

Effective dimensionality $d_{\text{eff}}(x, \tau)$ from MGD’s fitted θ_t

Generated with Claude Code, from code-reading and debugging sessions, 2026-09-23 to 2026-09-24

Purpose

Guth et al. define a local effective dimensionality $d_{\text{eff}}(x, \tau)$ of a distribution’s support, at a point x and noise scale τ , from the score of the variance-exploding (VE) noised distribution at x . This note derives d_{eff} directly in terms of MGD’s own fitted quantities – θ_t , φ , $\nabla\varphi$ – so it can be computed as pure post-processing of an already-completed MGD run, with no new sampling/fitting. Implementation: `codes/effective_dimension.py`. Notebook: `turbulence/effective_dimension_deff.ipynb`.

1 Guth’s d_{eff} : recap

Let q_τ be the law of $y = x + \sqrt{\tau}\xi$, $\xi \sim \mathcal{N}(0, I_d)$, x drawn from the data. Two equivalent definitions:

$$\text{energy form:} \quad d_{\text{eff}}(x, \tau) = d - \partial_s \mathbb{E}_y[U(y, \tau) \mid x], \quad s = \tfrac{1}{2} \log \tau, \quad (1)$$

$$\text{denoiser form:} \quad d_{\text{eff}}(x, \tau) = \frac{1}{\tau} \mathbb{E}_y[\|x - \mathbb{E}[x \mid y]\|^2 \mid x], \quad (2)$$

where $U(\cdot, \tau) = -\log q_\tau(\cdot)$ and, by Tweedie’s formula, $\mathbb{E}[x \mid y] = y + \tau \nabla_y \log q_\tau(y)$. Both reduce to d for an isotropic Gaussian and to (less than) d when the data lies near a lower-dimensional manifold, at scales τ finer than the manifold’s local curvature.

2 MGD’s schedule and sign convention

MGD’s Cos stochastic interpolant (`sde_routines.py`, `compute_interpolant`, `interpolant='Cos'` – the only schedule `forward_regularised` supports) is

$$I_t = \cos(\alpha_t) x_0 + \sin(\alpha_t) x_1, \quad \alpha_t = \frac{\pi t}{2}, \quad x_0 \sim \mathcal{N}(0, I_d), \quad x_1 = x \text{ (data)}. \quad (3)$$

Sign convention. This codebase fits θ_t for $p_\theta(x) \propto \exp(+\theta^\top \varphi(x))$ – the *opposite* of the MGD paper’s $\exp(-\theta^\top \varphi(x))$. This is audited directly against the code in `codes/utils_entropy.py` (“Sign convention audit”): the corrector step in `iteration_step_projection` is $x_{k+1} = y_k + \nabla \varphi^\top \theta_{\text{raw}}$ (a *plus*), giving a continuous-time drift $\sigma^2 \nabla \varphi^\top \theta_t$ that must equal $\sigma^2 \nabla \log p_t$ for the transport to preserve p_t , so

$$\nabla_x \log p_t(x) = +\theta_t^\top \nabla \varphi(x). \quad (4)$$

Independently corroborated by $dH_k = -\theta_k^\top \dot{m}_t$ (the entropy increment already computed in `iteration_step_projection`) and by $\log Z_{\theta_1} \gtrsim H_{\text{bound}} + \theta_1^\top m_1$ (MGD Eq. 8/30 as implemented in `utils_entropy.log_Z_bound`). Every formula below is re-derived directly in this (+) convention – not by substituting $\theta_{\text{paper}} = -\theta_{\text{code}}$ into the paper’s stated formulas.

3 The VE rescaling

Define $y_t := I_t / \sin(\alpha_t) = x_1 + \cot(\alpha_t) x_0$. Conditional on $x_1 = x$,

$$y_t \mid x \sim \mathcal{N}(x, \tau(t) I_d), \quad \tau(t) = \cot^2(\alpha_t), \quad (5)$$

monotonically decreasing from $\tau(0) = \infty$ to $\tau(1) = 0$. So MGD's Cos schedule already sweeps every VE noise scale, and t_{reg} (the grid θ_t is fit on) IS a τ -grid via Eq. 5.

Write q_τ (Guth's VE marginal, density in y) in terms of p_t (MGD's marginal, density in I_t) by the change of variables $I_t = \sin(\alpha_t) y$:

$$q_\tau(y) = \sin^d(\alpha_t) p_t(\sin(\alpha_t) y). \quad (6)$$

Differentiating and using Eq. 4 (evaluated at $I_t = \sin(\alpha_t) y$):

$$\nabla_y \log q_\tau(y) = \sin(\alpha_t) \nabla_x \log p_t(\sin(\alpha_t) y) = + \sin(\alpha_t) \theta_t^\top \nabla \varphi(\sin(\alpha_t) y). \quad (7)$$

The extra $\sin(\alpha_t)$ factor is the chain-rule term the derivation asked to double-check – confirmed, with the (+) sign from Eq. 4 (a paper-convention derivation would carry a minus here).

Practical note. $\sin(\alpha_t) y = \sin(\alpha_t) x + \cos(\alpha_t) \xi$ for $y = x + \sqrt{\tau} \xi$ (since $\sqrt{\tau} = \cot \alpha_t$) – literally `compute_interpolant`'s own Cos schedule with x as a single-point data endpoint and ξ as fresh noise. So every $\varphi/\nabla\varphi$ evaluation below stays inside the numerical range the fit itself already explores; nothing is evaluated off-distribution in an uncontrolled way.

4 Denoiser form

Substituting Eq. 7 into Tweedie's formula, $x - \mathbb{E}[x \mid y] = -\sqrt{\tau} \xi - \tau \nabla_y \log q_\tau(y)$, so the $1/\tau$ in Eq. 2 cancels analytically:

$$d_{\text{eff}}(x, \tau) = \mathbb{E}_\xi \left[\left\| \xi + \sqrt{\tau} \sin(\alpha_t) \theta_t^\top \nabla \varphi(\sin(\alpha_t) y) \right\|^2 \right], \quad y = x + \sqrt{\tau} \xi. \quad (8)$$

No $\log Z_t$, no m_t , no normalisation constant – only θ_t and $\nabla\varphi$. This is the primary estimator (`effective_dimension.denoiser_d_eff`).

5 Energy form

From $U(y, \tau) = -\log q_\tau(y)$,

$$U(y, \tau) = -\theta_t^\top \varphi(\sin(\alpha_t) y) + \log Z_t - d \log \sin(\alpha_t). \quad (9)$$

The last term is analytic in t ; the first two are not, so Eq. 1's ∂_s is taken numerically, by finite-differencing $\mathbb{E}_y[U(y, \tau) \mid x]$ directly across the t_{reg} grid (converted to $s = \frac{1}{2} \log \tau(t)$), rather than splitting analytic/Monte-Carlo pieces (`effective_dimension.energy_d_eff`).

log Z_t : a bound, not an exact value. MGD's Prop. 4.3/Eq. 30 gives, in this code's convention,

$$\log Z_t \gtrsim H_{\text{bound}}(t) + \theta_t^\top m_t, \quad H_{\text{bound}}(t) = H(p_0) + \int_0^t dH_s ds, \quad (10)$$

generalised from MGD’s stated $t = 1$ endpoint to any t (the integral is just truncated earlier). This is a *lower* bound on $\log Z_t$, so **the energy-form d_{eff} inherits that bound’s bias** – it is a secondary cross-check on the denoiser form, not an independent estimate. $H_{\text{bound}}(t)$ is built by trapezoid-integrating the already-saved dH_t^{bound} (`effective_dimension.h_bound_at`); m_t is either the saved `aux_moments` (the run’s own realised target moments) or, if not pulled from the cluster yet, recomputed directly from the data and schedule, $m_t = \mathbb{E}_Z[\varphi(\cos \alpha_t Z + \sin \alpha_t x_1)]$ – independent of any saved particle trajectory, so no SDE rerun either way (`effective_dimension.recompute_m_t`).

Empirical confirmation (2026-09-24, real 50-seed $d=256$ sweep). The bound-bias prediction above was checked against a real, full-resolution run (not a synthetic test): 50 seeds, real `aux_moments` pulled from the cluster, the full t_{reg} grid (197 nodes), 128 Monte-Carlo draws per node. Two signatures both hold, matching what a one-sided lower bound predicts (rather than the sign-inconsistent, chaotic pattern a residual implementation bug would produce):

- **Consistent sign.** The energy form comes out *lower* than the denoiser form in 74% of all (seed, point, τ) comparisons overall, reaching 100% at the coarsest end of the grid.
- **Growing magnitude.** The median gap grows from about -15 near $\tau \approx 50$ to about -87 near $\tau \approx 1.2 \times 10^{-3}$, with its spread growing alongside (median std across seeds rising from ~ 5 to $\sim 100+$ over the same range) – consistent with the bound’s slack accumulating as an (unlogged) Fisher-divergence-type residual along the transport, which generically grows with t .

Since the residual reflects how well the fitted score $\theta_t^\top \nabla \varphi$ matches the true score at each step, **a better-converged fit (finer t -grid, more potentials, better regularisation, ...) should directly tighten $H_{\text{bound}}(t)$ and shrink this gap** – it is a property of the fit’s quality, not a fixed offset.

6 Validation against a closed-form Gaussian test

To separate genuine implementation bugs from the expected bound bias above, both estimators were checked against an exact, independently-derived ground truth – no SDE, no fitting, no θ_t from any real run. For $x \sim \mathcal{N}(0, \Sigma)$, $y = x + \sqrt{\tau} \xi$, $q_\tau = \mathcal{N}(0, \Sigma + \tau I)$ exactly, so Tweedie’s formula gives (diagonal basis, $\Sigma = \text{diag}(\sigma_i^2)$):

$$d_{\text{eff}}(x, \tau) = \sum_i \frac{\tau x_i^2 + \sigma_i^4}{(\sigma_i^2 + \tau)^2}, \quad (11)$$

verified against brute-force $(\Sigma + \tau I)^{-1}$ linear algebra (agrees to $\sim 10^{-16}$) and its $\tau \rightarrow 0$ ($\rightarrow d$) and isotropic-expectation ($\rightarrow d\sigma^2/(\sigma^2 + \tau)$) limits. For *this* synthetic data MGD’s own p_t is also exactly Gaussian, so $\theta_t = -d/(2v(t))$ and $\log Z_t = \frac{d}{2} \log(2\pi v(t))$, $v(t) = \cos^2 \alpha_t + \sin^2 \alpha_t \sigma_{\text{data}}^2$, are exact too (single-coefficient potential $\varphi(x) = \overline{x^2}$) – feeding these directly into the *unmodified* `denoiser_d_eff/energy_d_eff` isolates implementation bugs with zero fitting noise and zero bound bias.

Using the real production grid ($d=256$, t_{reg} reconstructed from `nt=40000`, `n_subsample=200`): **denoiser_d_eff matched the oracle to $< 0.5\%$ relative error everywhere** (10 decades of τ , $|\theta|$ up to 512) – its implementation is correct. `energy_d_eff`, despite the exact inputs, showed errors up to 1320%, concentrated at the *coarsest* end of the grid.

6.1 Root cause: $s = \frac{1}{2} \log \tau$ is highly non-uniform, and it was differentiated naively

$\tau(t) = \cot^2(\alpha_t)$ diverges (logarithmically, in s) at both $t \rightarrow 0$ and $t \rightarrow 1$, so $s(t)$ is intrinsically badly stretched near the ends of the range *regardless of the t -schedule* – confirmed directly: a perfectly

uniform-in-t grid has an even worse max/min local- Δs ratio ($340\times$) than the actual Cos-schedule production grid ($67\times$, since bunching points near $t = 1$ partially compensates). Differentiating $\mathbb{E}_y[U]$ across this grid with a raw finite difference (`numpy.gradient`, which defaults to `edge_order=1`) has two compounding failure modes, both confirmed in isolation:

- (A) **Boundary truncation bias.** `edge_order=1` is only first-order accurate at the array’s first/last nodes – on a noiseless synthetic quadratic in s , this alone produces a 6.4% error at the boundary (exact at every interior node).
- (B) **Monte-Carlo noise amplification** (the dominant effect): $U(t)$ is itself MC-estimated, and dividing its noise by a small local Δs amplifies it. An n_{mc} sweep at the worst node showed the error shrinking monotonically as n_{mc} grew ($0.77 \rightarrow 0.29 \rightarrow 0.23$ over $4\text{k} \rightarrow 16\text{k} \rightarrow 32\text{k}$), confirming noise, not just truncation.

The natural first fix, `edge_order=2`, removes (A) exactly (boundary error drops to machine precision on the noiseless check) but **does not help, and can make the realistic case worse** (max relative error 17.9 vs. 10.4) – its wider stencil has larger coefficients under uneven spacing, amplifying (B) more than it fixes (A).

6.2 Fix: a local polynomial derivative

`energy_d_eff` now differentiates $\mathbb{E}_y[U]$ via `effective_dimension.local_poly_derivative`: at each node, a weighted-least-squares quadratic is fit through a window of $2h + 1$ *neighbouring indices* (shifted, not shrunk, at the array edges, so every node – including the boundary – gets a full window) and differentiated analytically. This fixes (A) exactly (machine precision on the noiseless check, any window size) and reduces (B) by averaging over several points rather than two. Validated on the Gaussian test ($h = 5$, the shipped default): overall median relative error dropped $\sim 6\times$ ($1.4\% \rightarrow 0.22\%$), and the fraction of the grid accurate to $< 1\%$ roughly doubled ($39\% \rightarrow 70\%$ of nodes, with the “becomes reliable” point moving from $t \approx 0.80$ down to $t \approx 0.41$). The few nodes at the very coarsest end of the grid remain the least reliable part of an energy-form curve on principle – no local model, however fit, is well-conditioned there – and should be read with that in mind; the denoiser form has no such issue, since it involves no differentiation step at all.

7 Dictionary

Guth et al.	MGD (this codebase)
VE noise scale τ	$\tau(t) = \cot^2(\alpha_t)$, $\alpha_t = \pi t/2$
score $\nabla_y \log q_\tau(y)$	$+\sin(\alpha_t) \theta_t^\top \nabla \varphi(\sin(\alpha_t)y)$
energy $U(y, \tau)$	$-\theta_t^\top \varphi(\sin(\alpha_t)y) + \log Z_t - d \log \sin(\alpha_t)$
$\log Z_\tau$	$H_{\text{bound}}(t) + \theta_t^\top m_t$ (<i>bound</i> , Eq. 10)
noised sample $y = x + \sqrt{\tau} \xi$	$\sin(\alpha_t) y = \text{compute_interpolant}(x, \xi)$
per- τ fit	$\theta_t \equiv \text{Theta_reg}[i]$, grid t_{reg}

8 What’s used from a saved run

- `Theta_reg`, shape (n_t, r) – the regularised θ_t trajectory (`codes/regularised_theta.py`); used exclusively (not the raw, finer θ_t) for a single self-consistent grid and to avoid a pre-fix off-by-one alignment issue present in some saved runs’ raw θ_t/dH_t_bound pairing.

- `treg` – `Thetareg`’s own coarse time grid; not saved for the $d=256$ runs on disk as of this writing, so it is reconstructed offline (`reconstruct_treg`) by replaying `SDE.forward.regularised`’s block-accumulation and near-node-pruning exactly, from the saved fine grid t alone – verified to reproduce the on-disk `Thetareg` length exactly.
- Each run’s `fitted_potentials/` – $\varphi, \nabla\varphi$ with the SAME pruned active-statistic set the fit used (some potentials auto-prune during `fit()`; skipping this step silently mis-indexes every θ coefficient).
- `dH_t_bound` + the fine grid t – integrated to $H_{\text{bound}}(t)$ (Eq. 10), needed only by the energy form.
- `aux_moments` (`barphi_e`) if pulled from the cluster, else m_t recomputed from the run’s own x_1 – needed only by the energy form.

9 Caveats

- **The energy form’s remaining disagreement with the denoiser form is, at this point, believed to be the expected bound bias** (Section 5’s empirical confirmation), not a residual implementation bug – Section 6’s controlled test found and fixed a genuine differentiation bug, and what is left afterward is sign-consistent and grows smoothly with t , the signature the bound predicts rather than of noise/bugs. This conclusion rests on one 50-seed sweep at one (σ, λ) setting, not a systematic sweep – treat it as the working hypothesis, not a closed case.
- θ_t becomes numerically extreme as $t \rightarrow 1$ ($\tau \rightarrow 0$): every seed inspected so far has one dominant, near-null-direction coefficient at the finest scale. This is harmless for the actual energy/score (both stay smooth off-support – checked directly), but $\theta_t^\top \varphi$ and $\theta_t^\top \nabla\varphi$ must be computed in double precision, not the float32 the values are stored in. The Gaussian test only exercised $|\theta|$ up to 512, well below the real runs’ $\sim 10^{16}$ near $t \rightarrow 1$ – this specific extreme-magnitude regime is *not* covered by the validation in Section 6 and remains a plausible contributor to disagreement at the very smallest τ , not yet isolated from the bound bias there.
- The finest available τ is set by how close to $t = 1$ the run’s schedule and `n_subsample` block reach, not by an arbitrary choice – see `codes/effective_dimension.py`’s `reconstruct_treg` for the exact grid actually available for a given run.

Implementation changelog

- *2026-09-23*: `denoiser_d_eff/energy_d_eff` cast θ_t to `x.dtype` for `potentials.grad()` but not to `x.device` – harmless on a single-device (CPU) run, but a hard crash on Jean Zay, where θ_t/m_t load onto CPU (`map_location='cpu'`) while `x`/the potentials live on cuda. Fixed: both are now moved to `x.device` inside each function.
- *2026-09-24*: the differentiation bug and fix described in Section 6 (`local_poly_derivative` replacing a raw `numpy.gradient` call in `energy_d_eff`).